

Qué es: un caso de cierre que junta todo el hilo conductor. Las métricas sirven para diagnosticar (Etapa 1), hay un servidor de desarrollo contra Unicorn (Etapa 1), puertos expuestos y fuerza bruta (Etapa 3), logs (Etapa 5), monitoreo (Etapa 6) y el docker-compose.yml (Trabajo 7). Los estudiantes leen el caso en el celular (hilo-conductor-redes/documentacion/caso-siga-inscripciones.html) y completan la planilla impresa en grupos de 2 o 3.

Para copiar en el pizarrón

Lunes	07:50	08:15	14:00
Pedidos/s que llegan	4	180	9
Pedidos/s atendidos	4	22	9
Latencia p95	0,3 s	14 s	0,4 s
CPU de la VM	12 %	28 %	14 %
Conexiones MySQL	3	3	3
Enlace (de 100)	30 Mbps	74 Mbps	33 Mbps

El enlace incluye el resto del edificio: unos 30 Mbps en el pico. La tasa de error no va en el pizarrón porque la calculan ellos en la pregunta 1.

Tiempos (60 min)

- **5'**: pizarrón, consigna, armar grupos y pasar el link.
- **10'**: lectura del caso.
- **15'**: partes 1 y 2 (diagnóstico y cuenta).
- **20'**: partes 3 y 4 (seguridad y propuesta).
- **10'**: puesta en común. Que dos grupos que eligieron opciones distintas en la pregunta 10 (nube o enlace) las defiendan.

Si falta tiempo, la 11, la 12 y la 13 se pueden terminar en casa.

Las partes 1 a 3 tienen respuesta cerrada; la parte 4 es abierta y se evalúa por la justificación.

Respuestas esperadas

1. Llegan 180 y se atienden 22: $(180 - 22) / 180 \approx 88\%$ sin atender.
2. **El proceso de la app.** Los atendidos quedan clavados en 22 mientras llegan 180, y el p95 sube a 14 s porque se arma una cola. No era la CPU (28 %, lejos del 100), tampoco MySQL (siempre 3 conexiones, la base estaba tranquila) ni el enlace (74 de 100, no estaba lleno).
3. `python app.py` es el servidor de desarrollo de Flask y atiende **un pedido a la vez**. Con 4 vCPU, un solo proceso usa como mucho un núcleo, o sea el 25 %. Encima pasa la mayor parte del tiempo esperando a MySQL, así que ni siquiera ese núcleo se llena. **La máquina estaba ociosa: comprar hardware no lo arreglaba.**
4. $1 \div 0,045 \approx 22$ **pedidos/s**, que coincide con el pizarrón. Es el techo de un proceso que atiende de a uno.
5. **Empeoraba.** Cada F5 es un pedido nuevo que se suma a la cola, y además vuelve a bajar la foto de 200 KB. Es lo mismo que pasó con Healthcare.gov, si vieron los casos fallidos.
6. $180 \times 250 = 45.000 \text{ KB/s} = 45 \text{ MB/s} \rightarrow 360 \text{ Mbps}$, $+ 30 = 390 \text{ Mbps}$. No alcanzan ni 100 ni 300. **Si arreglan solo la app, el cuello de botella pasa al enlace.**
7. $180 \times 30 = 5.400 \text{ KB/s} \rightarrow 43,2 \text{ Mbps} + 30 \approx 73 \text{ Mbps}$: **entra en 100**. Conclusión: achicar lo que se manda (caché de estáticos) no cuesta nada y rinde más que pagar un enlace más grande. Conecta con la clase de HTML contra JSON.
8. **3306:** cualquiera desde internet puede probar la base, y con la contraseña del ejemplo (que es pública) entra como root y lee, cambia o borra todo. Para cerrarlo: sacar el `ports`: de `mysql` para que quede solo en la red interna, poner una contraseña fuerte en una variable de entorno y que Secretaría entre por VPN o túnel SSH. **8080:** es el panel de administración de la base, abierto a internet. Hay que sacarlo de los puertos publicados o dejarlo accesible solo desde la red interna o por VPN. **Domingo:** una fuerza bruta como la de hydra en la Etapa 3, y **funcionó:** el `302` seguido de `GET /panel` es un login correcto, o sea que alguien entró como admin la noche anterior a la inscripción. Esto es un incidente: hay que cambiar la contraseña ya, revisar qué hizo en `/panel` (logs de la app y `general_log`, como en la Etapa 5) y si tocó cupos o inscripciones, y limitar los intentos (`limit_req` en Nginx o bloqueo después de N fallos). Si nadie se da cuenta de que el login funcionó, marcarlo en la puesta en común.

9. **Arquitectura (9):** internet → nginx (80/443, el único puerto publicado) → siga con Unicorn → mysql, todo en la red interna. Monitoreo de la Etapa 6: Prometheus, Grafana, cAdvisor y mysqld-exporter, con Grafana sin publicar o detrás de un login.
10. **Decisiones (10):** con Unicorn, $180 \div 22 \approx 8,2$, así que **9 workers**. Coincide con la regla de $2 \times \text{núcleos} + 1 = 9$ que vieron en la Etapa 1. **Nube o enlace:** las dos opciones se pueden defender. Si aplican el caché, el enlace de 100 alcanza y la plata puede ir a la nube por 48 h: más CPU para los workers, un enlace de 1 Gbps y SIGA deja de competir con el wifi del edificio (el riesgo es migrar la base antes y traerla de vuelta después). El enlace de 300 le sirve al edificio todo el año, pero **sin caché no alcanza** (390). Si eligen sin dar un número, no cuenta. **Peso:** caché de estáticos en Nginx (expires o Cache-Control), comprimir la foto, gzip.
11. **Alertas (11), por ejemplo:** p95 > 1 s durante 2 min; errores 5xx > 1 %; CPU > 80 %; enlace > 80 Mbps; conexiones MySQL cerca de max_connections; más de 20 logins fallidos por minuto desde una misma IP. Lo importante es que la tercera columna diga **qué hace** el administrador cuando salta.
12. **RNF (12), por ejemplo:** 200 pedidos/s (180 más un margen), p95 < 1 s, errores < 1 %. Si se supera, una página liviana de "sala de espera" o una portada reducida, como el fallback de Prime Day.
13. **Prueba (13):** prueba de carga con ab (la versión realista de la Etapa 1) contra una copia del sistema, por ejemplo ab -n 5000 -c 200, mirando Grafana mientras corre. No contra el sistema en producción y no en Killercoda.